

Notes on the Kalman Filter Changes

buhl.kalman.spread: centering, and the choice of $\mathbf{P}$, R , $\mathbf{Q}$

Dante Buhl

October 2026

Summary

The state-space model is

$$\begin{aligned}\ln P_t^A &= \alpha_t + \beta_t \ln P_t^B + v_t, & v_t &\sim N(0, R), \\ \alpha_t &= \alpha_{t-1} + w_t^\alpha, & \beta_t &= \beta_{t-1} + w_t^\beta, & (w_t^\alpha, w_t^\beta) &\sim N(0, \mathbf{Q}).\end{aligned}$$

The filter is initialized by least squares on the first quarter of the series ($t < t_f$, `fmt` in the code) and then run forward over the rest. The changes are:

1. $\ln P^B$ is centered on the mean of the initial window. This is an exact change of variables, so the spread and β_t do not change, and α_t and $\mathbf{P}$ are converted back before returning (Section 1).
2. R , the residual variance of the initial fit, is unchanged (Section 2).
3. $\mathbf{P}_0$, the covariance matrix of the initial least-squares coefficients, is now built in the centered variables, and an indexing bug is fixed (Section 3).
4. $\mathbf{Q} = \Delta t \text{diag}(\sigma_\alpha^2, \sigma_\beta^2)$, a random walk $d\alpha = \sigma_\alpha dW$, $d\beta = \sigma_\beta dW$. The `np.diag` call is fixed, σ_α and σ_β are now function arguments, and σ_β defaults to 0.01 (Section 4).
5. Fixes to the recursion: indexing, array shapes, and the spread is now returned (Section 5).

All numbers below are for the pair SSD/LIN (basic materials, 2015–2023, $\Delta t = 1/252$), which passed the sector sweep.

1 Centering $\ln P^B$

Change. Before fitting, subtract the mean of $\ln P^B$ over the initial window, $\bar{x} = \frac{1}{t_f} \sum_{t < t_f} \ln P_t^B$, and run the filter in the variable $x_t = \ln P_t^B - \bar{x}$.

Why it does not change the data. For any (α, β) ,

$$\alpha + \beta \ln P_t^B = (\alpha + \beta \bar{x}) + \beta (\ln P_t^B - \bar{x}) = \tilde{\alpha} + \beta x_t, \quad \tilde{\alpha} = \alpha + \beta \bar{x}.$$

The model is identical with the intercept relabeled. The hedge ratio β and the spread $S_t = \ln P_t^A - \alpha - \beta \ln P_t^B$ are unchanged. In matrix form the change of variables is

$$\begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \mathbf{T} \begin{bmatrix} \tilde{\alpha} \\ \beta \end{bmatrix}, \quad \mathbf{T} = \begin{bmatrix} 1 & -\bar{x} \\ 0 & 1 \end{bmatrix},$$

so a covariance matrix of $(\tilde{\alpha}, \beta)$ maps back as $\mathbf{P} = \mathbf{T} \tilde{\mathbf{P}} \mathbf{T}^T$. Both conversions are applied before returning, so the outputs satisfy the same identity as `fit_spread`: $\ln P^A = \alpha + \beta \ln P^B + S$.

Why it helps. With $\ln P^B \approx 5.15$ and little variation, α is the intercept at $\ln P^B = 0$, far from the data. Raising β and lowering α fits almost equally well, so the two estimates are almost perfectly negatively correlated. After centering, $\tilde{\alpha}$ is the fitted level of $\ln P^A$ at the typical value of $\ln P^B$, and it is uncorrelated with β :

Initial covariance $\mathbf{P}_0$	raw $\ln P^B$	centered
$\text{corr}(\alpha, \beta)$	-0.99990	0.00000
condition number	1.2×10^5	2.4×10^2

```
# center lPb on the initial window, this is only a change of variables:
# alpha + beta*lPb = (alpha + beta*lPb_mean) + beta*(lPb - lPb_mean)
lPb_mean = np.mean(lPb[:fnt])
x = lPb - lPb_mean

# initial fit on reduced timeseries
S, hedge_ratio, other_stuff, result = fit_spread(lPa[:fnt], x[:fnt])
...
# undo the centering, alpha -> alpha - beta*lPb_mean and P -> T P T~T
alpha = alpha - hedge_ratio*lPb_mean
T = np.array([[1., -lPb_mean], [0., 1.]])
P = T@P@T.T
```

Numerical check. `fit_spread` on the initial window with raw and centered $\ln P^B$ gives spreads that agree to 1.8×10^{-14} , the same β , SSR and ADF p -value, and $\tilde{\alpha} - \beta \bar{x} = \alpha_{\text{raw}} = -1.087732$. I also wrote a separate filter entirely in the raw variables, with the equivalent $\mathbf{Q}_{\text{raw}} = \mathbf{T} \mathbf{Q} \mathbf{T}^T$, and compared it with the new `kalman_spread`:

$(\sigma_\alpha, \sigma_\beta)$	$\max \Delta \alpha_t $	$\max \Delta \beta_t $	$\max \Delta S_t $	$\max \Delta \mathbf{P} $
(0,0)	2.2×10^{-12}	4.7×10^{-13}	1.1×10^{-13}	5.0×10^{-16}
(0.03,0.01)	1.7×10^{-12}	3.6×10^{-13}	5.1×10^{-14}	3.7×10^{-14}
(0.1,0.1)	8.3×10^{-13}	1.6×10^{-13}	4.5×10^{-14}	6.7×10^{-14}

The two agree to rounding error. Centering changes the numerics, not the results.

One caveat. Centering does not change the data, but it does change what a *diagonal* $\mathbf{Q}$ means. A diagonal $\mathbf{Q}$ in the centered variables equals $\mathbf{Q}_{\text{raw}} = \mathbf{T} \mathbf{Q} \mathbf{T}^T$ in the raw variables, which is not diagonal. Putting the same σ 's on the diagonal in the raw variables is a different model:

$(\sigma_\alpha, \sigma_\beta) = (0.03, 0.01)$	half-life (days)	std(S_t)	range of β_t
diagonal $\mathbf{Q}$, centered (new)	18.8	0.0726	0.933 – 1.091
diagonal $\mathbf{Q}$, raw (old)	12.2	0.0593	0.906 – 1.049

The centered version is the meaningful one. There σ_α is the drift of the spread’s level, which is not tied to the arbitrary point $\ln P^B = 0$.

2 Observation noise R

Change. None to the formula. It was already in place before this change (replacing the original placeholder $R = 0$): R is the residual variance of the initial least-squares fit,

$$R = \frac{\text{SSR}}{t_f - 2},$$

dividing by $t_f - 2$ because two coefficients are estimated.

Why. In the observation equation the noise v_t is the spread itself, i.e. the part of $\ln P^A$ not explained by $\alpha_t + \beta_t \ln P_t^B$. Its variance is the stationary variance of the spread over the initial window. For SSD/LIN, $\sqrt{R} \approx 0.10$, which matches the static spread’s standard deviation. With $R = 0$ the filter treats every observation as exact and puts the entire error into (α, β) .

```
# R: variance of the observation noise, residual variance of initial fit
R = res/(fnt-2)
```

`res` is the sum of squared residuals returned by `fit_spread` (now `resid[0]`, a scalar). Centering does not change it.

3 Initial state covariance $\mathbf{P}_0$

Change. The formula, the covariance matrix of the least-squares coefficients from the initial fit, was already in place. It is now built from the centered x_t , which makes it diagonal, and the slicing bug is fixed.

Why. $\mathbf{P}$ is the filter’s uncertainty in (α, β) . At initialization we have exactly that from regression: with design matrix $\mathbf{A}$ (rows $[1, x_t]$, $t < t_f$) and residual variance R ,

$$\text{Var} \begin{bmatrix} \hat{\alpha} \\ \hat{\beta} \end{bmatrix} = R(\mathbf{A}^T \mathbf{A})^{-1}.$$

The diagonal holds $\text{Var}(\hat{\alpha})$ and $\text{Var}(\hat{\beta})$, which is the “diagonal with the error of the initial fit” idea made exact. The off-diagonal term is kept. In the centered variables it is zero anyway (see Section 1), so $\mathbf{P}_0$ is diagonal there. For SSD/LIN, $\text{sd}(\hat{\beta}) \approx 0.066$ at initialization.

```
# P: covariance of (alpha, beta) from the initial least-squares fit
A = np.stack((np.ones(fnt), x[:fnt]), axis=1)
P = R*np.linalg.inv(A.T@A)
del A
```

This also fixes `1Pb[:, fnt]`, which indexed a column of a 1-D array. It should be the slice `1Pb[:fnt]`, now the centered `x[:fnt]`.

4 State noise $\mathbf{Q}$

Change. $\mathbf{Q}$ was originally $\mathbf{I} \Delta t$. It is now

$$\mathbf{Q} = \Delta t \begin{bmatrix} \sigma_\alpha^2 & 0 \\ 0 & \sigma_\beta^2 \end{bmatrix},$$

with σ_α and σ_β passed as arguments (defaults 0.03 and 0.01 per $\sqrt{\text{year}}$).

Why. $\mathbf{Q}$ is the noise in the evolution of (α, β) . Modeling them as Brownian motion is right, but the general form is $d\beta_t = \sigma_\beta dW_t$, whose per-step variance is $\sigma_\beta^2 \Delta t$. $\mathbf{Q} = \mathbf{I} \Delta t$ is the special case $\sigma_\alpha = \sigma_\beta = 1$, i.e. *standard* Brownian motion. With $\Delta t = 1/252$ that means β wanders about ± 1 per year and ± 0.06 per day. For a hedge ratio near 1, that would make the hedge meaningless. The filter then attributes almost every price move to a change in β , and the spread's half-life collapses to about 0.2 days.

A smaller σ is still a random walk, just with a smaller diffusion coefficient. $\sigma = 0$ recovers the static hedge ratio, so the σ 's interpolate between the static model of Problem 3 and a hedge that adapts every day:

σ_β (per $\sqrt{\text{yr}}$)	$q = \sigma_\beta^2 \Delta t$	typical drift of β in 1 year
1 (old)	4×10^{-3}	± 1
0.05	1×10^{-5}	± 0.05
0.01	4×10^{-7}	± 0.01
0	0	none (static)

Choosing σ_α . After centering, $\tilde{\alpha}$ is the spread's level, so its drift competes directly with the mean reversion we want to trade. Over one half-life τ the level wanders by about $\sigma_\alpha \sqrt{\tau}$. That should be small compared with the spread's stationary standard deviation. For SSD/LIN (std ≈ 0.10 , $\tau \approx 0.17$ yr), $\sigma_\alpha = 0.03$ gives a wander of ≈ 0.012 , which is small. $\sigma_\alpha = 0.1$ gives ≈ 0.04 , which absorbs a large part of the spread.

Grid on SSD/LIN (centered; entries are innovation log-likelihood / half-life in days / std of spread):

$\sigma_\alpha \backslash \sigma_\beta$	0	0.01	0.03	0.1
0	2796 / 43.5 / 0.109	3098 / 33.4 / 0.097	3366 / 20.6 / 0.079	3585 / 10.3 / 0.058
0.01	3131 / 33.1 / 0.095	3215 / 28.8 / 0.090	3398 / 19.8 / 0.077	3589 / 10.2 / 0.058
0.03	3436 / 19.4 / 0.074	3448 / 18.8 / 0.073	3502 / 16.0 / 0.068	3611 / 9.8 / 0.055
0.1	3644 / 8.7 / 0.051	3644 / 8.7 / 0.051	3647 / 8.5 / 0.050	3665 / 7.1 / 0.046
0.3	3662 / 3.4 / 0.034	3662 / 3.4 / 0.034	3662 / 3.4 / 0.033	3659 / 3.3 / 0.033

The likelihood keeps rising toward large σ 's, but there the half-life falls to a few days: the filter is fitting the noise. The likelihood favors large σ because the model assumes v_t is white when it is actually an OU process. The defaults (0.03, 0.01) keep a tradeable half-life (≈ 19 days) while letting the hedge adapt. This trade-off is the answer to Problem 4(b)'s question "closer to stationary, or just fitting noise?". The final values should be chosen on the formation period and checked out of sample in Problem 5.

```
def kalman_spread(lPa, lPb, dt, sigma_alpha=0.03, sigma_beta=0.01):
    ...
    # Q: covariance of the random walk increments of (alpha, beta) per step
    Q = np.diag([sigma_alpha**2, sigma_beta**2])*dt
```

This also fixes `np.diag(sigma_alpha**2, sigma_beta**2)`. `np.diag` takes one array, and its second argument is the diagonal offset, so passing a scalar raised `ValueError: Input must be 1- or 2-d`. The previous default $\sigma_\beta = 0.05$ is lowered to 0.01 based on the grid above.

5 Fixes to the recursion

```
for i in range(fnt, nt):
    H = np.array([1., x[i]])
    # propagate the uncertainty forward
    P = P + Q
    # compute the error in the prediction, this is the spread at time i
    # using only alpha, beta known at time i-1
    err = lPa[i] - alpha[i-1] - hedge_ratio[i-1]*x[i]
    # compute uncertainty of observations
    OE = H@P@H + R
    # Kalman Gain
    K = P@H/OE
    # Update alpha, hedge_ratio, uncertainty
    alpha[i] = alpha[i-1] + K[0]*err
    hedge_ratio[i] = hedge_ratio[i-1] + K[1]*err
    P = (np.eye(2) - np.outer(K, H))@P
    spread[i] = err
```

- **Indexing.** The loop used `i+1` up to `nt-1`, so the last step raised an `IndexError`. It now updates step `i` from step `i-1`. The first step uses the initial fit stored at `fnt-1`.
- **Shapes.** `H` is a 1-D array, so `H@P@H` is a scalar and `K` is a length-2 vector. There are no 1×1 arrays or nested lists.
- **Spread returned.** The spread is the one-step prediction error $e_t = \ln P_t^A - \alpha_{t-1} - \beta_{t-1} \ln P_t^B$. It uses only information available at time t , so it can be traded in Problem 5 without lookahead. For $t < t_f$ it is the in-sample spread of the initial fit. The *updated* residual $\ln P_t^A - \alpha_t - \beta_t \ln P_t^B$ is not used, because it is close to zero almost by construction.
- **Return value** is now `[alpha, hedge_ratio, spread, P]` in the original variables. Before, it returned only the final `P` and no spread.

6 Verification

Synthetic data. With $\ln P^B$ a random walk and $\ln P^A = -0.5 + 1.3 \ln P^B + \text{noise}$ (constant coefficients, 2000 steps), the filter with $\sigma_\alpha = \sigma_\beta = 0$ ends at $\alpha = -0.508$, $\beta = 1.302$ ($\text{sd}(\beta) = 0.0024$). With the defaults it ends at $\alpha = -0.494$, $\beta = 1.298$.

SSD/LIN, static vs. Kalman (filtered part of the formation period):

	β	half-life (days)	std of spread
static OLS (full period)	1.019	51.4	0.111
Kalman (0.03, 0.01)	0.933 – 1.091	18.8	0.073

The check script is `check_kalman.py`. It compares the centered filter with a raw-variable reference, the old diagonal $\mathbf{Q}$, and the synthetic case.